

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №4

Тема: Рефакторинг коду та проведення Code Review
за індустріальними стандартами

Посилання на Git-репозиторій: <https://github.com/SPEAKER3/X-View/>

Результати Code Review одногрупника:

№	Рядок коду	Проблема	Категорія	Рекомендація
1	ExtractionJob.cs (рядки 11, 27, 37, 46)	Використання жорстко заданих рядків для статусів ("Pending", "Completed") робить код крихким і схильним до помилок (опечаток).	Magic Numbers / Magic Literals	Створити enum JobStatus (зі значеннями Pending, Processing, Completed, Failed) та змінити тип властивості Status. Також int maxRetries = 3; варто винести як private const int MaxRetries = 3; на рівень класу.
2	ExtractionJob.cs (рядок 60)	Ініціалізація нового об'єкта Random при кожному виклику методу може призвести до генерації однакових значень	Resource Mismanagement	Необхідно зробити цей об'єкт статичним полем класу.
3	Document.cs (рядки 8, 10) та ExtractionJob.cs (рядок 46)	Рядкові властивості (FileName, Format, PromptText) можуть бути null при виході з конструктора.	Uninitialized properties / Nullable reference types	Необхідно додати модифікатор required до цих властивостей.

	10)		danger	
4	ExtractionJob.cs (метод ExecuteWithLLMAsync, весь блок циклу while)	Метод ExecuteWithLLMAsync с перевантажений обов'язками.	Long Method	Рекомендується винести логіку циклу while в окремий приватний метод (наприклад, ExecuteWithRetryAsync), щоб покращити читабельність та спростити підтримку коду.
5	ExtractionJob.cs (блоки try та catch, де викликається Console.WriteLine)	Використання Console.WriteLine безпосередньо у класах бізнес-логіки є поганою практикою	Exception Handling / Side Effects	Для логування слід впровадити інтерфейс ILogger через Dependency Injection, або ж просто передавати інформацію про помилки вище за стеком викликів через винятки.

Посилання на Github Issues: <https://github.com/SPEAKER3/X-View/issues>

Результати рефакторингу власного коду:

БУЛО:

```
public string FileName { get; set; }
public string Format { get; set; }
```

СТАЛО:

```
public string FileName { get; set; } = string.Empty;
public string Format { get; set; } = string.Empty;
```

ЧОМУ:

Рядкові властивості за замовчуванням могли мати значення null, що порушувало правила безпеки типів (Nullable reference types) і могло призвести до `NullReferenceException`. Ініціалізація порожнім рядком (`string.Empty`) вирішує цю проблему і прибирає попередження літера.

БУЛО:

```
public string Status { get; set; } = "Pending";  
// ...  
Status = "Completed";  
Status = "Failed";
```

СТАЛО:

```
public enum JobStatus { Pending, Processing, Completed, Failed }  
// ... в класі ExtractionJob:  
public JobStatus Status { get; set; } = JobStatus.Pending;  
// ... далі в кодї:  
Status = JobStatus.Completed;  
Status = JobStatus.Failed;
```

ЧОМУ:

Використання жорстко захардкоджених текстових рядків для статусів легко призводить до помилок (опечаток). Використання типізованого переліку `enum` гарантує безпеку під час компіляції та покращує читабельність логіки.

БУЛО:

```
private async Task<string> MockLLMCallAsync(string prompt)  
{  
    var rnd = new Random();  
    if (rnd.Next(0, 10) < 5) ...  
}
```

СТАЛО:

```
// На рівні класу додано статичне поле:  
private static readonly Random _rnd = new Random();
```

```
private async Task<string> MockLLMCallAsync(string prompt)
{
    if (_rnd.Next(0, 10) < 5) ...
}
```

ЧОМУ:

Створення нового екземпляра Random при кожному виклику методу створювало зайве навантаження на Garbage Collector. Винесення Random у статичне readonly-поле класу (_rnd) оптимізує споживання пам'яті.

Звіт регресійного тестування:

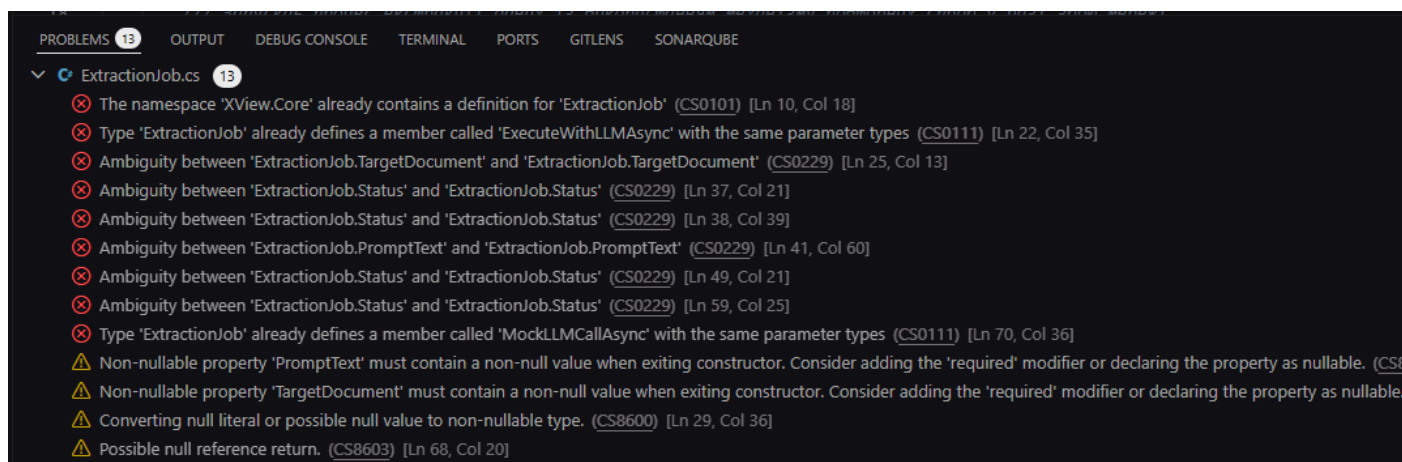
```
Восстановление завершено (0,3 с)
XView.Tests net10.0 успешно выполнено (0,2 с) → bin\Debug\net10.0\XView.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.07]   Discovering: XView.Tests
[xUnit.net 00:00:00.12]   Discovered:   XView.Tests
[xUnit.net 00:00:00.14]   Starting:    XView.Tests
[xUnit.net 00:00:00.21]   Finished:   XView.Tests
XView.Tests (тест) net10.0 успешно выполнено (0,8 с)

Сводка теста: всего: 18; сбой: 0; успешно: 18; пропущено: 0; длительность: 0,8 с
Сборка успешно выполнено через 1,7 с

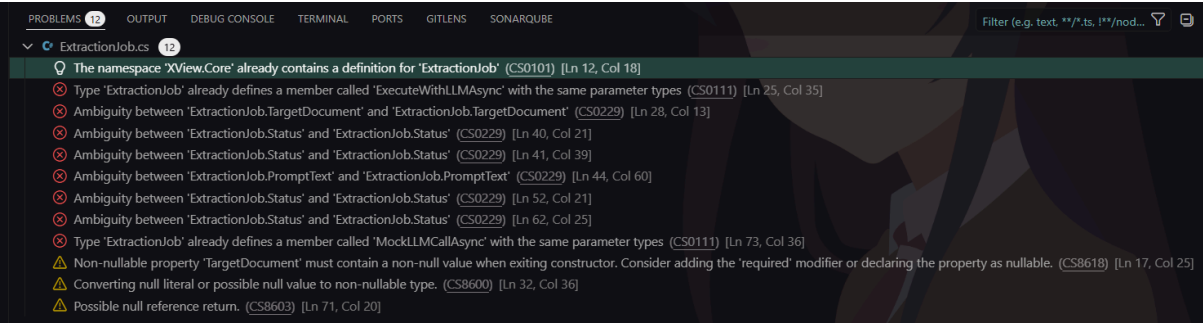
C:\Users\avior\Desktop\SHARAGA\PRG_D\X-View\XView.Tests>dotnet build
Восстановление завершено (0,3 с)
XView.Tests net10.0 успешно выполнено (0,2 с) → bin\Debug\net10.0\XView.Tests.dll

Сборка успешно выполнено через 0,9 с
```

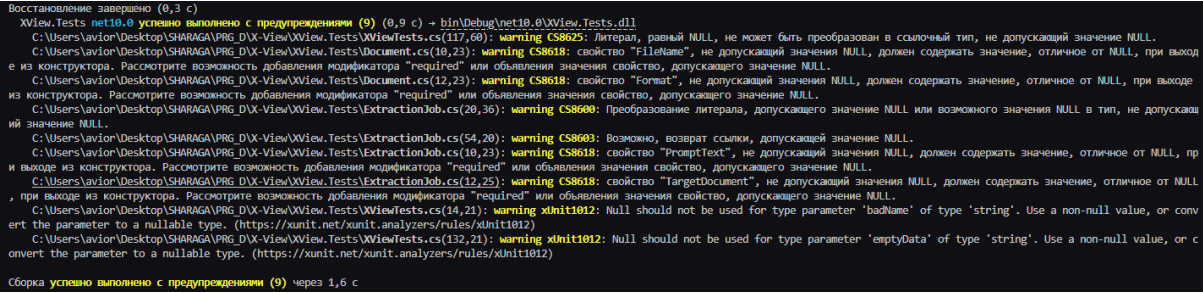
Порівняння метрик ДО



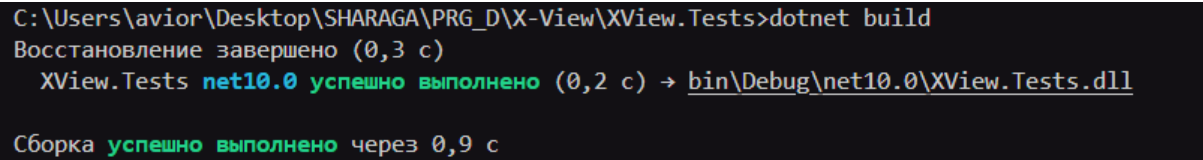
ПІСЛЯ



ДО



ПІСЛЯ



Цикломатична складність:

NLOC	CCN	token	PARAM	length	location		
15	5	59	0	20	Document::Validate@20-39@.\Document.cs		
31	7	220	1	50	ExportFile::GenerateCSV@20-69@.\ExportFile.cs		
34	5	131	0	42	ExtractionJob::ExecuteWithLLMAsync@14-55@.\ExtractionJob.cs		
8	2	46	1	9	ExtractionJob::MockLLMCallAsync@57-65@.\ExtractionJob.cs		
6	1	48	1	11	DocumentTests::Validate_EmptyFileName_ShouldThrowArgumentException@17-27@.\XViewTests.cs		
6	1	50	1	11	DocumentTests::Validate_ValidFileSize_ShouldNotThrowException@33-43@.\XViewTests.cs		
7	1	60	0	12	DocumentTests::Validate_FileSizeExceedsLimit_ShouldThrowInvalidOperationException@47-58@.\XViewTests.cs		
6	1	52	1	11	DocumentTests::Validate_ValidFormat_ShouldNotThrowException@64-74@.\XViewTests.cs		
6	1	50	1	11	DocumentTests::Validate_InvalidFormat_ShouldThrowNotSupportedException@80-90@.\XViewTests.cs		
7	1	67	0	13	ExtractionJobTests::ExecuteWithLLMAsync_DocumentIsValid_ThrowsExceptionBeforeExecution@98-110@.\XViewTests.cs		
6	1	48	0	11	ExtractionJobTests::ExecuteWithLLMAsync_DocumentIsNull_ThrowsNullReferenceException@114-124@.\XViewTests.cs		
6	1	38	1	11	ExportFileTests::GenerateCSV_EmptyData_ShouldThrowArgumentNullException@134-144@.\XViewTests.cs		
7	1	41	0	12	ExportFileTests::GenerateCSV_InvalidJsonSyntax_ShouldThrowFormatException@148-159@.\XViewTests.cs		
7	1	39	0	12	ExportFileTests::GenerateCSV_EmptyJsonArray_ShouldReturnEmptyString@163-174@.\XViewTests.cs		
12	1	70	0	17	ExportFileTests::GenerateCSV_ValidJsonArray_ShouldReturnCorrectCsvString@178-194@.\XViewTests.cs		
7	1	37	0	12	ExportFileTests::GenerateCSV_ValueWithComma_ShouldWrapInQuotes@198-209@.\XViewTests.cs		
7 file analyzed.							
NLOC	Avg.NLOC	Avg.CCN	Avg.token	function_cnt	file		
3	0.0	0.0	0.0	0	.\obj\Debug\net10.0\NETCoreApp,Version=v10.0.AssemblyAttributes.cs		
9	0.0	0.0	0.0	0	.\obj\Debug\net10.0\XView.Tests.AssemblyInfo.cs		
8	0.0	0.0	0.0	0	.\obj\Debug\net10.0\XView.Tests.GlobalUsings.g.cs		
25	15.0	5.0	59.0	1	.\Document.cs		
41	31.0	7.0	220.0	1	.\ExportFile.cs		
55	21.0	3.5	88.5	2	.\ExtractionJob.cs		
126	6.9	1.0	50.0	12	.\XViewTests.cs		
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)							
Total nloc	Avg.NLOC	Avg.CCN	Avg.token	Fun Cnt	Warning cnt	Fun Rt	nloc Rt
267	10.7	1.9	66.0	16	0	0.00	0.00

Висновки: У результаті виконання роботи закріплено практичні навички з проведення інспекції коду, використання систем контролю версій для трекінгу проблем (Issues) та виконання безпечного рефакторингу.

Статичний аналіз: Налаштовано інструмент SonarQube for IDE (SonarLint) та використано вбудовані аналізатори Roslyn (C#). Це дозволило автоматизовано виявити початкові проблеми коду, пов'язані з безпекою типів та стилем написання.

Code Review: У процесі аналізу вихідного коду (у форматі взаємоперевірки) було ідентифіковано та задокументовано у вигляді GitHub Issues 5 архітектурних недоліків (code smells). Серед них: використання «магічних рядків», небезпека нульових посилань (Nullable reference types) та неефективне управління ресурсами пам'яті.

Рефакторинг та регресійне тестування: Виконано рефакторинг коду для усунення знайдених проблем. Використано типізовані переліки (enum), додано ініціалізацію значень за замовчуванням та оптимізовано роботу з класом Random. Наступний запуск модульних тестів (18/18 пройдено успішно) підтвердив, що рефакторинг не порушив існуючу бізнес-логіку системи.

Оцінка якості: Повторний запуск лінтерів зафіксував повну відсутність попереджень. Аналіз за допомогою метрик складності показав, що код став більш надійним та оптимізованим без збільшення цикломатичної складності методів. Додатково було успішно застосовано ШІ-асистента (GitHub Copilot) для аналізу та документування патерну повторних спроб (Retry Pattern).